タマゴインキュベーター C担当（演出・フィードバック）実装仕様書

対象: チームD「タマゴインキュベーター」当日実装（8/29・8/30） C担当 松崎正弘 参照元: 開発計画_フィードバック反映版／タマゴインキュベーター_クラス図／0826_Memo（ノンバーバル表現表）／目標設計ワークシート_松崎正弘（以上Google Drive チームDフォルダ）／卵演出_本番統合ガイド.md（以下「統合ガイド」、ユーザー提供） 検証状況: 本仕様書のUnity側の記述（フィールド名・シーン階層）は、統合ガイドの記載を鵜呑みにせず、実際に検証用Unityプロジェクト（EggBreakController.cs/EggHatchController.cs/EggCrackGlowController.cs/GameManager.csが存在するプロジェクト）にMCP経由で接続し、スクリプトのソースコードとシーンヒエラルキーを直接読んで一致を確認済み。

---

0. この仕様書の使い方

この1ファイルだけを当日AIに渡せば、C担当の実装範囲が過不足なく分かることを目標にしている。統合ガイド（既存3スクリプトの詳細な使い方）はこの仕様書が前提として参照するが、統合ガイド自体も当日一緒に渡すこと（3〜7章の細部はそちらが一次情報）。

バイブコーディングにしないための使い方

自由AI（Cursor無料枠／Gemini無料枠／ChatGPT無料枠等、レギュレーション上有料AI不可）に丸投げで「これ実装して」と投げると、既存の3スクリプト（EggBreakController/EggHatchController/EggCrackGlowController）を無視して同じ機能を再実装してしまったり、dangerLevelの意味（5章参照）を取り違えたまま実装してしまうリスクが高い。そのため、AIに依頼するときは次の順番を守ること。

1. まず統合ガイドの1〜2章と、本仕様書の2章・5章を読ませ、**「なぜこの設計になっているか」を自分の言葉で説明させてから**実装させる（説明できないなら理解できていない）
2. 1コンポーネントずつ依頼する（3章の単位で区切る）。「C担当全部作って」は避ける
3. 生成されたコードは、必ず4章の依存関係表と2.4節のAPI早見表に照らして、フィールド名・呼び出し先が実在するものと一致しているかを人間が確認する
4. Unity実機で動作確認するまでは「実装できた」と判断しない（8章のチェックリストを使う）
5. 詳細は9章にまとめてある

---

1. スコープの再確認：何が完成済みで、何をこれから作るか

チームDの開発計画上、C担当の実装スコープは以下の11項目だった。今回、事前にBlender/Unityで**ひび割れ発光・落下破壊・孵化の3演出（卵の見た目そのもの）が実装・検証済み**という状況を踏まえ、当日の実装範囲を次のように再整理する。

開発計画上のC担当スコープ	状態	当日の作業
孵化演出（段階的なヒビ→割れ→キャラ登場）	**実装・検証済み**（EggHatchController）	本番卵への組み込みのみ（統合ガイド2章）
ひび割れ・光の視覚演出	**実装・検証済み**（EggCrackGlowController）	危険度の入力元を接続するのみ（5章）

開発計画上のC担当スコープ	状態	当日の作業
落下時の卵が割れる表現	**実装・検証済み**（`EggBreakController`）	本番卵への組み込みのみ（統合ガイド2章）
心音の動的制御（BGM/SE）	未実装	新規実装（3.2節 `HeartbeatController`）
バイブレーションのパターン制御	未実装	新規実装（3.3節 `HapticController`）
カメラ制御（固定/追従/シェイク）	未実装	新規実装（3.4節 `CameraController`）
BGM/SE実装（タップ・成功・失敗等）	一部フック済み（破損音・孵化音の再生スロットのみ）	新規実装（3.5節 `AudioController`）
視覚エフェクト（タップ・転がり等）	未実装（ヒビ・破損・孵化の主要エフェクトは既存アセット側で対応済み）	新規実装（3.6節 `EffectController`、範囲を縮小）
タイトル・リザルト画面、UI実装	未実装	新規実装（3.7節 `UIController`）
背景・ライティング調整	未実装	シーン作業（本仕様書の対象外、Unity上で直接調整）
操作方法を伝える導入演出	未実装	新規実装（`UIController`または専用スクリプト）

つまり「卵そのものの見た目の変化」はすでに完成しており、当日C担当が新規に書くコードは、既存の卵演出を外側から呼び出す／音・振動・カメラを付け足す「統括レイヤー」が中心になる。 これは2章のクラス図で言う `FeedbackController` とその配下（`HeartbeatController`/`HapticController`/`CameraController`/`AudioController`/`EffectController`/`UIController`）にそのまま対応する。

---

2. 既存アセット（3スクリプト）の要点

詳細は統合ガイド1〜3章・7章を参照。ここでは当日の実装に必要な最小限だけをまとめる。

2.1 役割分担

スクリプト	発火条件	卵はどうなるか
`EggCrackGlowController`	常時。`dangerLevel`（0〜1）に応じて殻が光る	見た目が変わるだけ。メッシュ切替はなし
`EggBreakController`	卵が台から落下し地面等に衝突した瞬間（失敗）	殻→8破片に切り替わり弾ける
`EggHatchController`	制限時間を耐えきった瞬間（成功）	予兆（揺れ+発光）→殻→薄い8破片+キャラクターに切替、穏やかに弾ける

落下破壊と孵化はゲーム上排他的な結末だが、2つのコンポーネントはコード上疎結合であり、排他制御は呼び出し側（ゲームロジック）の責任（統合ガイド1.1節）。6章で扱う。

2.2 実際のシーン構成（検証用Unityプロジェクトで直接確認済み）

```
Egg_Test                          ← Rigidbody（1個のみ）/ EggCrackGlowController /
EggBreakController / EggHatchController
 ├─ Egg_RadialGlow                 ← 放射光条（EggCrackGlowControllerが自動検出）
 ├─ EggFragments                   ← 落下破壊用8破片（Egg_Frag_01〜08）
 └─ EggHatch（初期非アクティブ）    ← 孵化用一式
     ├─ Character                   ← 孵化キャラ（Animatorでアニメーション再生、現状は仮モ
デル）
     └─ HatchFragments              ← 孵化用の薄い8破片（HatchShell_01〜08）
```

本番の卵Prefabも、この階層構造（原点・回転・スケールを卵と揃える）を踏襲する（統合ガイド2.3節Step 3、6.2節）。

2.3 `dangerLevel` の実体（ソースコードで確認した重要事実）

`EggCrackGlowController.cs` の実装を直接読んで確認したところ、このスクリプトは**将来的に心音テンポと同期する前提で設計されている**ことが分かった。

```
[Header("心音テンポ同期（将来拡張）")]
[Tooltip("有効にすると危険度をベースに心拍(ドクン、ドクン)の明滅を重ねる")]
public bool enableHeartbeatSync = false;
[Tooltip("心拍テンポ（BPM）。危険度が上がるほど外部からこの値を上げる想定")]
public float heartbeatBPM = 70f;
[Tooltip("心拍パルスがどれだけ発光を強めるか（0=無効, 1=最大2倍まで明滅）")]
public float heartbeatPulseStrength = 0.35f;
```

つまり `dangerLevel` は「殻の光り方」だけでなく、**将来的に音の心音と視覚の明滅を同じテンポで同期させる**ことを見越したフィールドである。これはC担当が実装する `HeartbeatController`（3.2節）と自然に連携できる設計になっているため、5章で具体的な接続方針を示す。

2.4 公開API早見表（統合ガイド4.2節・7章と同一、C担当が実際に呼ぶもの）

クラス	呼ぶメソッド／プロパティ	いつ呼ぶか
`EggCrackGlowController`	`SetDangerLevel(float 0〜1)`	危険度・進行度が更新されるたび（毎フレーム可）
`EggCrackGlowController`	`testAutoPlay = false`	ロジック制御を始める前に**必ず1回**（1.3節）
`EggCrackGlowController`	`enableHeartbeatSync` / `heartbeatBPM`	任意。5章参照
`EggBreakController`	`TriggerBreak()`	落下確定時（自動衝突発火 or ロジック駆動、6章）
`EggBreakController`	`onBreak`（UnityEvent）	ここにC担当の音・振動・カメラ拡張を登録（統合ガイド5章）
`EggHatchController`	`TriggerHatch()`	制限時間終了時
`EggHatchController`	`onHatch`（UnityEvent）	ここにC担当の音・振動・カメラ拡張を登録

3. C担当が新規実装するコンポーネント仕様

`FeedbackController` を起点に、既存の3スクリプトを「呼び出す側」として統括する。クラス図（8/26作成）の設計をベースに、既存アセットとの接続方法を明記する。

3.0 全体構成

```
GameManager（B担当）
    │  danger / progress / result を通知
    ▼
FeedbackController（C担当・統括）
    ├─ HeartbeatController   … 危険度→心音の音程・テンポ（新規）
    ├─ HapticController      … 振動パターン（新規）
    ├─ CameraController      … 固定/追従/シェイク（新規）
    ├─ AudioController       … BGM・汎用SE（新規）
    ├─ EffectController      … タップ等の補助VFX（新規・範囲縮小）
    ├─ UIController           … タイトル/リザルト/UI（新規）
    └─（直接参照）EggCrackGlowController / EggBreakController / EggHatchController（既存・実装済み）
```

`FeedbackController` は既存の卵演出3スクリプトへの参照も持ち、`UpdateDangerFeedback()` の中で `eggCrackGlowController.SetDangerLevel()` を呼ぶ。落下・孵化の**トリガー自体（`TriggerBreak`/`TriggerHatch`の呼び出し）はB担当のゲーム進行ロジックが行う**想定（6章）。`FeedbackController`はそれに付随する音・振動・カメラだけを担当する。

3.1 `FeedbackController`（統括）

項目	内容
役割	GameManagerからの通知を各サブコントローラへ振り分ける。既存の卵演出3スクリプトへの参照も保持
フィールド	`heartbeatController` `hapticController` `cameraController` `effectController` `audioController` `uiController`（各サブコントローラ）／`eggCrackGlowController`（既存スクリプトへの参照）
メソッド	`UpdateDangerFeedback(float danger)` … `heartbeatController.UpdateHeartbeat(danger)` と `hapticController.PlayHeartbeat(danger)` を呼び、`eggCrackGlowController.SetDangerLevel()` は5章の合成ロジックで別途更新する `UpdateCrackFeedback(float crackProgress)` … 5章の合成に使う値を保持・更新する `PlaySuccessFeedback()` … `hapticController.PlaySuccess()` `cameraController.PlaySuccessCamera()` `audioController.PlaySuccess()` `uiController.ShowSuccessResult()` を呼ぶ（`TriggerHatch()`自体はここでは呼ばない、6章） `PlayFailureFeedback()` … 同様に失敗側を呼ぶ（`TriggerBreak()`自体はここでは呼ばない） `PlayEggMovementFeedback(float speed)` … `audioController.PlayRolling(speed)` `effectController.PlayRolling(speed)`

項目	内容
実装ノート	`Awake()`で`eggCrackGlowController.testAutoPlay = false;`を必ず実行する（統合ガイド1.3節）。忘れると危険度制御が効かない

3.2 `HeartbeatController`（新規実装・音の心音）

項目	内容
役割	危険度に応じて心音AudioSourceのピッチ・再生テンポを変化させる
フィールド	`heartbeatSource: AudioSource`／`basePitch: float`（既定1.0）／`maxPitch: float`（既定1.8程度）／`baseInterval: float`（心音の再生間隔・秒）／`minInterval: float`
メソッド	`StartHeartbeat()` … BGMとは別トラックでループ再生開始 `UpdateHeartbeat(float danger)` … `danger`（0〜1、**位置ベースの危険度**、5章参照）に応じて`heartbeatSource.pitch`を`basePitch`〜`maxPitch`で線形補間し、再生間隔も`baseInterval`〜`minInterval`で補間して再スケジュールする `StopHeartbeat()` … 停止（成功/失敗確定時）
ノンバーバル表現表との対応	安定=心音なし/かなり弱い、危険軽度=ゆっくり、危険中度=速い、危険最大=かなり速い（0826_Memoより）
実装ノート	入力は**B担当が算出する位置ベースの`danger`（台の中心〜端の距離）**を使う。孵化進行度（時間経過）は使わない（5章で理由を説明）

3.3 `HapticController`（新規実装・振動）

項目	内容
役割	危険度に応じた継続的な振動パターンと、割れ/孵化時の単発振動を制御する
メソッド	`PlayHeartbeat(float danger)` … `danger`に応じて`Handheld.Vibrate()`相当の短い振動を心音と同じ間隔で発生させる（Android想定。強さの段階制御が必要な場合はAndroidネイティブAPI経由のプラグイン検討、無ければ`Handheld.Vibrate()`の呼び出し頻度で強弱を表現） `PlayCrack()` … ヒビが一定段階を超えた瞬間の短い振動（5章の閾値検知と合わせて使用） `PlaySuccess()` … 孵化成功時の気持ちよい強めの短い振動。`eggHatchController.onHatch`に登録（統合ガイド5.2節と同じ手順） `PlayFailure()` … 落下・破損時の強い一発。`eggBreakController.onBreak`に登録
実装ノート	`PlaySuccess()`/`PlayFailure()`は統合ガイド5.1〜5.2節の「拡張フックの登録方法」をそのまま踏襲する。**`EggBreakController.cs`/`EggHatchController.cs`自体は一切改変しない**

3.4 `CameraController`（新規実装）

項目	内容
役割	通常時の固定カメラ、失敗時の追従、危険時・成功/失敗時のシェイク
フィールド	`mainCamera: Camera`
メソッド	`PlayNormal()` … 初期の固定アングルに戻す `PlayDangerShake(float danger)` … 危険度が高い間、微小な揺れを加える(統合ガイド5.1節の`CameraShake`実装例を流用可) `FollowFallingEgg(Transform egg)` … **卵が「台から落下したと確定した瞬間」（＝`EggBreakController.TriggerBreak()`が呼ばれるより前、A/B担当の落下確定判定のタイミング）に呼ぶ**。統合ガイド5章の`onBreak`（衝突＝着地の瞬間）はこれより後なので、`onBreak`にはシェイクなど「衝撃の一押し」だけを登録する `PlaySuccessCamera()` … 孵化成功時の演出カメラ（`eggHatchController.onHatch`に登録） `ResetCamera()` … リトライ時に`PlayNormal()`へ戻す
実装ノート	追従開始と衝撃シェイクの**タイミングが別**であることに注意（表4章参照）。落下確定の通知はB担当のEggFallDetector相当から受け取る

3.5 `AudioController`（新規実装・BGM/汎用SE）

項目	内容
役割	BGM切り替えと、心音以外のSE（タップ・転がり・開始等）の再生
フィールド	`bgmSource: AudioSource`／`seSource: AudioSource`／各種`AudioClip`（タイトルBGM/ゲーム中BGM/成功リザルトBGM/失敗リザルトBGM/タップSE/転がりSE/開始SE）
メソッド	`PlayBGM(BGMType type)`／`PlaySE(SEType type)`／`PlayRolling(float speed)` … 速度に応じて音量・ピッチを変化 `PlaySuccess()` / `PlayFailure()` は**基本的に呼ばなくてよい**。既存の`EggHatchController.hatchAudioSource`/`hatchSfx`、`EggBreakController.breakAudioSource`/`breakSfx`にInspectorで直接クリップを割り当てれば、成功/失敗の効果音は既存スクリプトが単独で鳴らせる（統合ガイド2.4節チェックリスト）。`AudioController`側で二重に鳴らさないよう注意

項目	内容
実装ノート	「割れ音」「孵化音」は既存アセットのフィールドに直接割り当てるのが最も手数が少ない。`AudioController`はBGMの切り替えと、それ以外のSEに専念させることで実装量を削減する

3.6 `EffectController`（新規実装・範囲を縮小）

項目	内容
役割	ヒビ・破損・孵化の主要VFXは既存アセット側（パーティクル・シェーダー）が担うため、**それ以外の補助エフェクトのみ**を担当する
メソッド	`PlayRolling(float speed)` … 卵が転がっている間の砂煙・軌跡等（あれば）
実装ノート	当初のクラス図では`UpdateCrack()`/`PlayHatch()`等を`EffectController`が持つ設計だったが、**その役割は既存の`EggCrackGlowController`/`EggHatchController`が既に持っているため重複実装しない。**`EffectController`はタップ演出などプラスアルファのみに縮小する

3.7 `UIController`（新規実装）

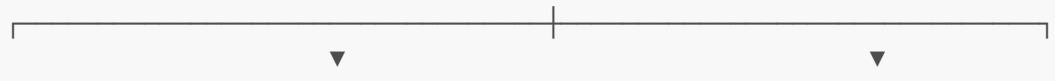
項目	内容
役割	タイトル・プレイ中・リザルト・リトライのUI切り替え
メソッド	`ShowTitle()` `ShowPlaying()` `ShowSuccessResult()` `ShowFailureResult()` `ShowRetry()`
実装ノート	「操作方法を伝える導入演出（テキストを用いない誘導）」もここに含める。テキストを使わない前提のため、フェードイン/アイコンアニメーション等で表現する

---

4. データフロー全体とA・B担当との接続点

```
[A担当 EggController]                    [B担当 GameManager/DifficultyManager/TimerManager]
  卵の位置・速度                              進行度(progress)・危険度(danger)・
  落下検知                                    ヒビ進行度(crackProgress)・成否(result)
        │ speed, eggTransform                       │ danger, progress, crackProgress, result
        ▼                                           ▼
 FeedbackController.PlayEggMovementFeedback()
FeedbackController.UpdateDangerFeedback(danger)

FeedbackController.UpdateCrackFeedback(crackProgress)
                                                    │
```



```
        HeartbeatController.UpdateHeartbeat(danger)
HapticController.PlayHeartbeat(danger)   eggCrackGlowController.SetDangerLevel(合成値, 5章)
```

受け取るデータ	提供元	用途
`danger`（位置ベース危険度 0〜1）	B担当	`HeartbeatController`／`HapticController`／`CameraController.PlayDangerShake` の入力。かつ5章の合成値の一部
`progress`（経過時間の進行度 0〜1）	B担当	UIの残り時間表示、5章の合成値の一部
`crackProgress`	B担当（実装するかは6章の議論次第）	使わない可能性あり（5章参照）
`result`（成功/失敗）	B担当	`PlaySuccessFeedback()`/`PlayFailureFeedback()`の呼び出しトリガー
`speed`（卵の速度）	A担当 `EggController.GetSpeed()`	`PlayEggMovementFeedback(speed)`
落下確定の通知（着地前）	A/B担当（`EggFallDetector` 等）	`CameraController.FollowFallingEgg(eggTransform)` の起動タイミング（3.4節）

仮値運用: B担当の実装が完了する前は、`danger`・`progress`にダミー値（Inspectorのスライダー、または簡易コルーチンで0→1へ変化させる仮スクリプト）を使って`FeedbackController`単体で動作確認する。8/29午前はこの仮値ベースで先行着手する（7章）。

---

5. 設計判断：`dangerLevel` に何を渡すか（要B担当確認）

チームのノンバーバル表現表（0826_Memo）は、本来「位置ベースの危険度（6段階）」と「時間経過によるヒビの進行（3段階）」という**2つの独立した軸**を想定していた。しかし既存の`EggCrackGlowController`は`SetDangerLevel(float)`という**単一の入力**しか持たない（2.3節）。この差分をどう埋めるかが、当日最初に決めるべき設計判断になる。

推奨方針：2つの軸を1つの値に合成して渡す。

```
float visualCrack = Mathf.Max(danger, progress);
eggCrackGlowController.SetDangerLevel(visualCrack);
```

- `danger`（位置ベース）が高ければ、時間が浅くても殻はすぐ強く光る＝「今すぐ危ない」を反映
- `progress`（時間経過）が進めば、位置が安定していても徐々に光る＝「もうすぐ生まれる」を反映
- どちらか高い方を採用する（`Max`）ことで、両方の意味を1チャンネルに矛盾なく載せられる

一方、**`HeartbeatController`（音）と`HapticController`（振動）の心音系は`danger`（位置ベース）のみを入力とする**（3.2節）。理由は、ノンバーバル表現表で心音のテンポ変化が明確に「危険・軽度〜最大」という位置ベースの

状態と紐づけられているため。時間経過の意味は視覚（殻の発光）側に寄せ、音・振動は「今の危険」に集中させることで、2つのチャンネルの役割を重複させずに済む。

要確認事項：progressとcrackProgress（B担当が別途持つヒビ段階値）が同じものを指すのか別物なのかは、B担当の実装次第で変わる。8/29午前にB担当と、①dangerの値域・更新頻度、②progress（またはcrackProgress）が上記の合成に使える形になっているか、をすり合わせること（7章のコミュニケーション計画）。

ヒビが入った瞬間の単発振動（HapticController.PlayCrack()）について：既存アセットには「ヒビが1段階増えた」というイベントは無く、visualCrackは連続値である。そのため、C担当側のUpdate内でvisualCrackが0.33、0.66等の閾値をまたいだ瞬間を自前で検知し、そのタイミングでPlayCrack()を呼ぶ形で近似する（既存スクリプトの改変は不要）。

---

6. 成功／失敗の確定と排他制御（要A・B担当確認）

統合ガイド1.1節・4.6節の通り、EggBreakControllerとEggHatchControllerはコード上疎結合であり、**「落下破壊が起きたのに直後にタイマーも切れて孵化も呼ばれる」ような競合を防ぐ排他制御は、呼び出し側が実装する責任**になっている。

推奨方針：この排他制御とTriggerBreak()/TriggerHatch()の呼び出し自体は、開発計画上「ゲーム進行・状態管理」を担当するB担当のGameManager側に置く（統合ガイド4.6節のEggOutcomeManagerパターンをB担当のGameManagerに組み込む形）。C担当のFeedbackControllerは、TriggerBreak()/TriggerHatch()を**呼ばれた結果として**、onBreak/onHatchイベント経由で自分の演出（音・振動・カメラ）だけを差し込む。

```
// B担当 GameManager 側（イメージ）
public void NotifyFellOff()
{
    if (_resolved) return;
    _resolved = true;
    cameraController.FollowFallingEgg(egg.transform); // C担当のカメラを先に始動
    eggBreakController.TriggerBreak();                // 着地の瞬間に殻が割れる
    feedbackController.PlayFailureFeedback();          // 音・振動・UI
}

public void NotifyTimeUp()
{
    if (_resolved) return;
    _resolved = true;
    eggHatchController.TriggerHatch();
    feedbackController.PlaySuccessFeedback();
}
```

要確認事項：この「誰がTriggerBreak/TriggerHatchを呼ぶか」の役割分担は、B担当の設計次第で変わりうる（例えばEggStateControllerという別クラスが持つ可能性もクラス図上ある）。8/29午前のコアループ実装のタイミングで、A・B担当と**「排他制御はどのクラスの責任か」を明示的に合意する**こと。合意した結果に応じて、FeedbackController側の呼び出しコードだけを調整すればよい設計にしてあるため、影響範囲は小さい。

---

7. 実装順序（当日スケジュールとの対応）

前回まとめた「当日実装計画」（別紙）の時系列と対応させる。本仕様書に基づく作業内容として読み替える。

8/29午前：B担当の仮値（danger・progress）を使い、HeartbeatControllerを実装。あわせてFeedbackControllerの土台（3.1節）と、既存3スクリプトを本番卵Prefabへ組み込む作業（統合ガイド2.3節

Step1〜5、8章チェックリスト）を進める。

8/29午後：`HapticController`・`CameraController`を実装し、5章の合成ロジックで`eggCrackGlowController.SetDangerLevel()`を接続。`onBreak`/`onHatch`への音・振動・カメラのフック登録（統合ガイド5.2節の手順）を行う。各節目でAndroid実機ビルドし、A・B担当と一緒に確認する。

8/30：新規実装は行わず、8章のチェックリストに沿って組み込み漏れがないか確認し、初見テストプレイのフィードバックをもとに、5章の合成係数や`HeartbeatController`のピッチ/テンポの数値だけを調整する。

---

8. 組み込み・動作確認チェックリスト

統合ガイド2.4節をベースに、C担当の新規実装分を加えたもの。

既存アセットの組み込み

- ☐ Rigidbodyは卵の最上位に1つだけ（子オブジェクトに重複していないか、統合ガイド6.1節）
- ☐ `shellWhole`/`fracturedPieces`/`hatchVisual`等の参照がInspectorで全て埋まっている
- ☐ 破片群のローカル座標が卵と同一原点・回転なし・スケール1（統合ガイド6.2節）
- ☐ 破片の各メッシュに`MeshCollider(Convex ON)`+`Rigidbody(isKinematic ON)`が付いている
- ☐ `EggCrackGlowController.testAutoPlay`が`false`になっている（`FeedbackController.Awake()`で自動設定、3.1節）
- ☐ 台（フライパン）側のColliderが`Ground`タグを持っている

C担当の新規実装分

- ☐ `HeartbeatController`：危険度0→1で心音のピッチ・テンポが自然に変化する
- ☐ `HapticController.PlayHeartbeat`：危険度に応じて振動間隔が変化する
- ☐ `HapticController.PlaySuccess`/`PlayFailure`が`onHatch`/`onBreak`から正しく発火する（二重発火していないか）
- ☐ `CameraController.FollowFallingEgg`が、着地（`onBreak`）より前、落下確定のタイミングで始動している
- ☐ `eggCrackGlowController.SetDangerLevel()`に5章の合成値が渡っており、位置・時間の両方で殻が光ることを確認
- ☐ `breakAudioSource`/`breakSfx`、`hatchAudioSource`/`hatchSfx`に本番SEを割り当てた（未設定だと無音のまま、統合ガイド2.4節）
- ☐ 危険度6段階すべてに、ノンバーバル表現表どおりの視覚・音・振動が入っている（実機で確認）

---

9. AIに実装させる際の使い方（詳細）

0章の要約に加えて、当日の具体的な進め方。

1. **最初にこの仕様書と統合ガイドを両方渡し、「1章のスコープ表を見て、今日新規に書くべきクラスを列挙して」と聞く。** 列挙結果が3章の7クラスと一致しなければ、まだ理解が浅いので読み直させる
2. **1クラスずつ、3章の該当セクションをそのままプロンプトに含めて依頼する。**「フィールド」「メソッド」「実装ノート」を守らせ、勝手にAPIを増減させない
3. **`EggBreakController.cs`/`EggHatchController.cs`/`EggCrackGlowController.cs`を絶対に改変させない**（onBreak/onHatchへのInspector登録、またはコードからの`AddListener`のみで拡張する。統合ガイド5章）
4. 生成後、4章の表と2.4節のAPI早見表を照らし合わせ、存在しないメソッド名やフィールド名を使っていないか人間が確認する
5. 5章・6章は「未決定・要相談」の設計判断であることをAIにも明示し、勝手に確定させない（合意前は仮の値・仮のクラス構成で進めてよいが、変更しやすい形にしておく）

---

10. 未決事項（統合ガイド8章を継承・当日中に確認）

- 演出フック（`breakParticles`/`breakAudioSource`/`breakSfx`、`hatchParticles`/`hatchAudioSource`/`hatchSfx`）は現状すべて未設定（統合ガイド8章）→本番SE・パーティクル割り当てが必要
- 落下破壊と孵化の排他制御の実装者（6章、要A・B確認）
- `danger`と`progress`（または`crackProgress`）の値域・更新頻度の仕様（5章、要B確認）
- 破片8個という分割数がプレイフィールに合っているか（統合ガイド8章、要実機調整）
- Android実機でのBloomの見え方（統合ガイド8章、PC Editorと差が出る可能性あり）
- キャラクターモデルは仮のプレースホルダー（統合ガイド3.2節、正式素材への差し替えとライセンス確認が必要）
- `HapticController`の振動強度制御方法（`Handheld.Vibrate()`は単純なON/OFFのみのため、強弱パターンを作るには頻度制御か、Androidネイティブプラグインの検討が必要）

---

11. 出典

- チームDフォルダ（Google Drive）：`開発計画_フィードバック反映版.docx`／`タマゴインキュベーター_クラス図.drawio.png`／`0826_Memo`／`目標設計ワークシート_松崎正弘.xlsx`／`Android実機ビルド・テスト手順_タマゴインキュベーター.docx`
- ユーザー提供資料：`卵演出_本番統合ガイド.md`
- 検証用Unityプロジェクト（MCP経由で直接確認）：`Assets/Scripts/EggBreakController.cs`／`EggHatchController.cs`／`EggCrackGlowController.cs`／`GameManager.cs`、およびシーン`SampleScene`のヒエラルキー（`Egg_Test`以下の構造）